

Proyecto Final

Lowest Common Ancestor (LCA)

Equipo:

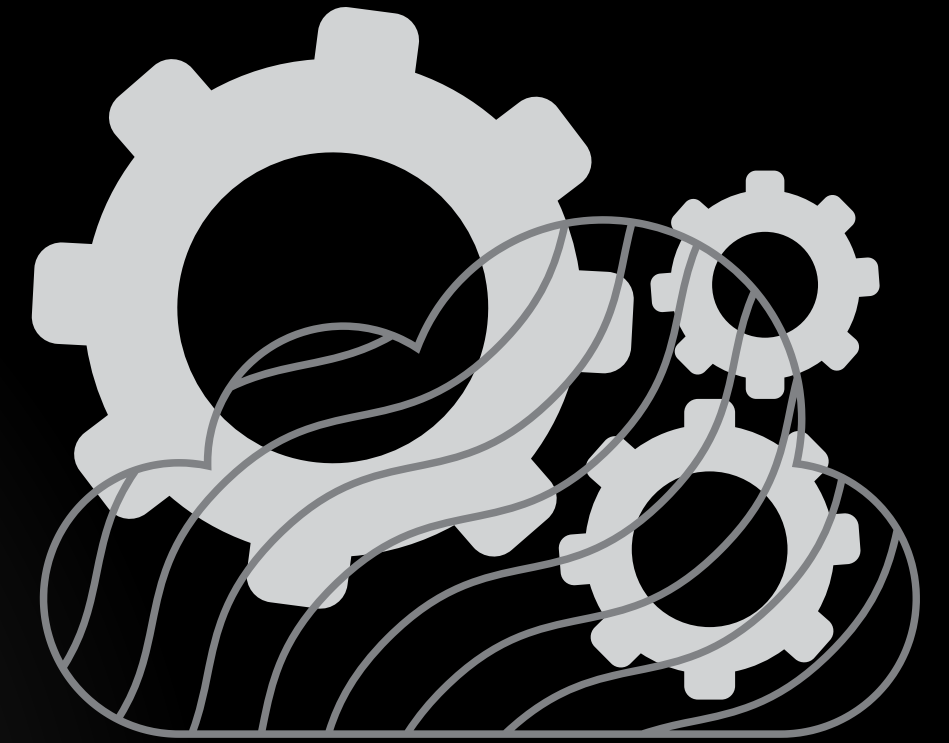
- Nombre: IA
- Integrantes:
 - Ismael Gándara Cornejo
 - Angel Miguel Villalvazo Vazquez

Materia: Análisis De Algoritmos

Sección: D01

Profesor: Jorge Ernesto López Arce Delgado

Fecha: 26 de noviembre del 2025



UNIVERSIDAD DE
GUADALAJARA

Red Universitaria e Institución Benemérita de Jalisco

Centro Universitario de Ciencias Exactas e Ingenierías
Departamento Ciencias Computacionales
Ingeniería en Computación

EL PROBLEMA

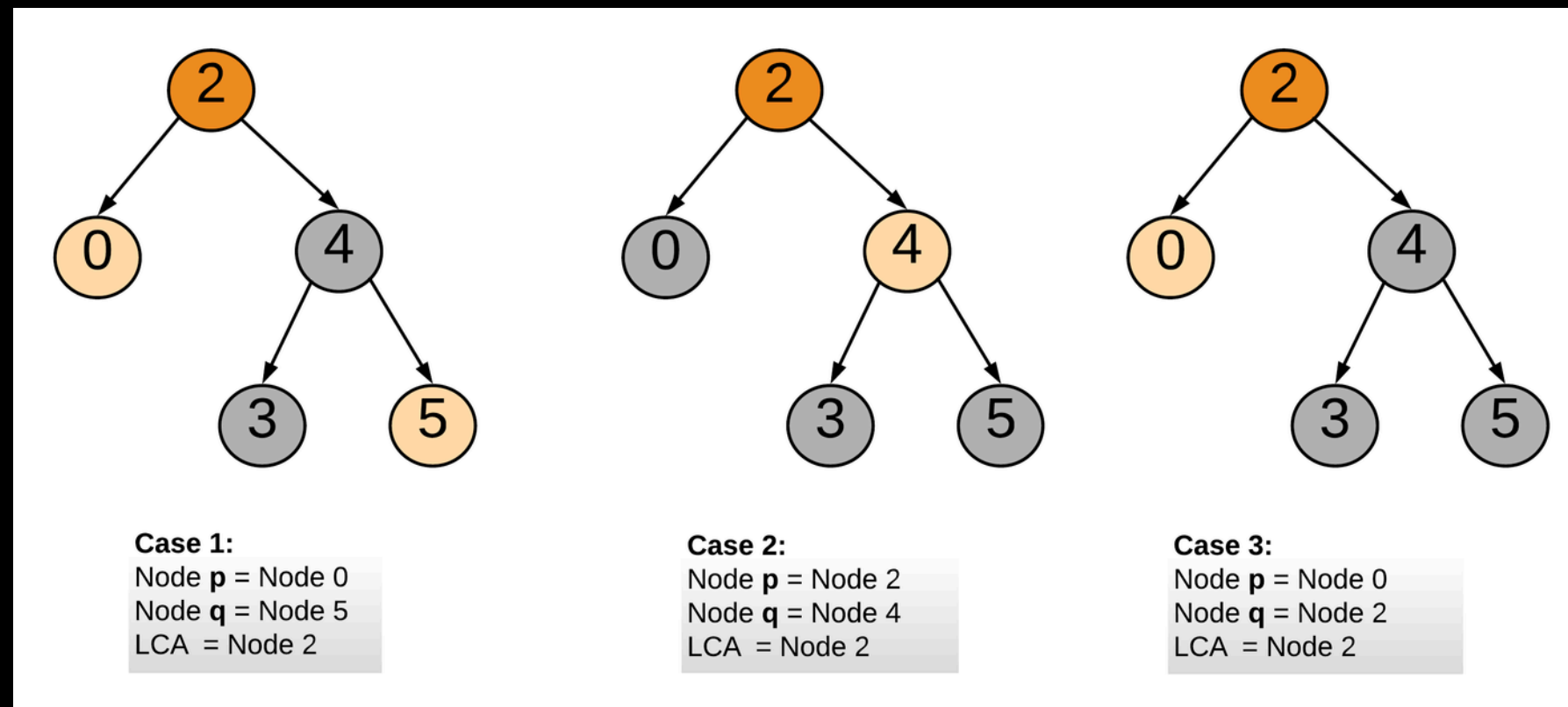
El Ancestro Común Más Bajo (LCA) determina la jerarquía y relación entre los nodos dentro de la estructura de un árbol. Se aplica, en la práctica, en sistemas de control de versiones (git), bioinformática (árboles filogenéticos) y redes de computadoras.

Para esto desarrollamos un programa en Python que calcula el LCA mediante las técnicas de Fuerza bruta, Binary Lifting de divide y vencerás con programación dinámica, y Tarjan, visualizando paso a paso su ejecución. También se integra una función para la compresión de datos basada en Huffman, aplicando así los algoritmos voraces.

ALGORITMO BASE

Dado un árbol y dos nodos u y v , el objetivo es encontrar el nodo más profundo que sea ancestro de ambos.

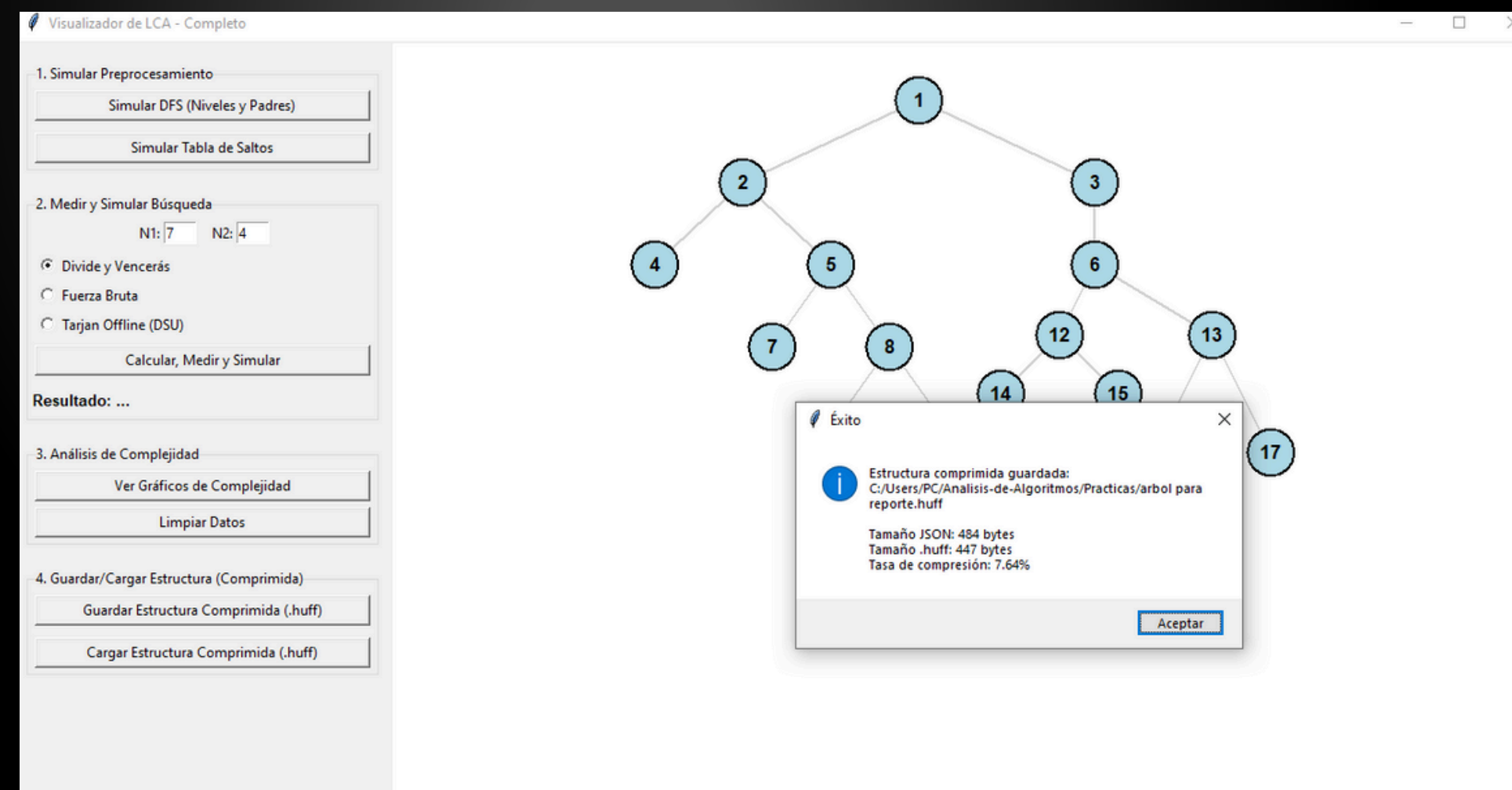
- **Reto de Eficiencia:** En árboles muy grandes, recorrer desde los nodos hasta la raíz repetidamente es ineficiente.
- **Reto de Almacenamiento:** Guardar estructuras de grafos complejas en texto plano (JSON) puede ser costoso en espacio.



COMPRESIÓN DE HUFFMAN

Asigna códigos binarios de diferente tamaño a los caracteres, basándose en la frecuencia de aparición. Los caracteres más frecuentes obtienen códigos más cortos. En este proyecto lo utilizamos para comprimir el JSON que define la estructura del árbol.

Para la Implementación utilizamos la librería heapq y con esto construimos el árbol de frecuencias y luego se genera un archivo .huff para guardar el árbol y después poder cargarlo. Al guardar, el sistema muestra la tasa de compresión.

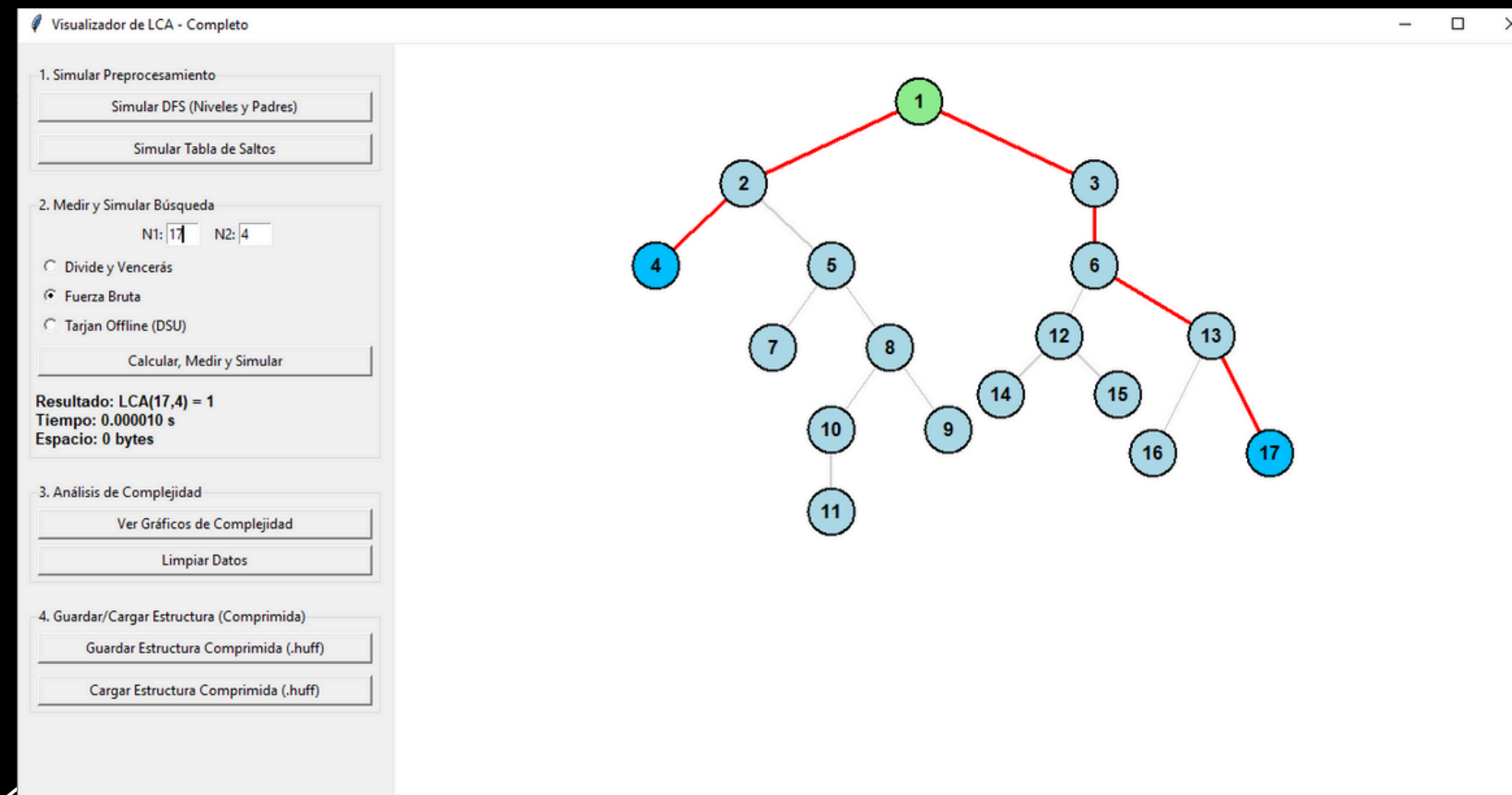


FUERZA BRUTA

Primero se igualan los niveles de ambos nodos moviendo el más profundo hacia arriba. Luego, ambos nodos suben simultáneamente nivel por nivel hasta encontrarse.

El punto donde se encuentran es el LCA y tiene una complejidad de búsqueda de $O(N)$ en el peor caso.

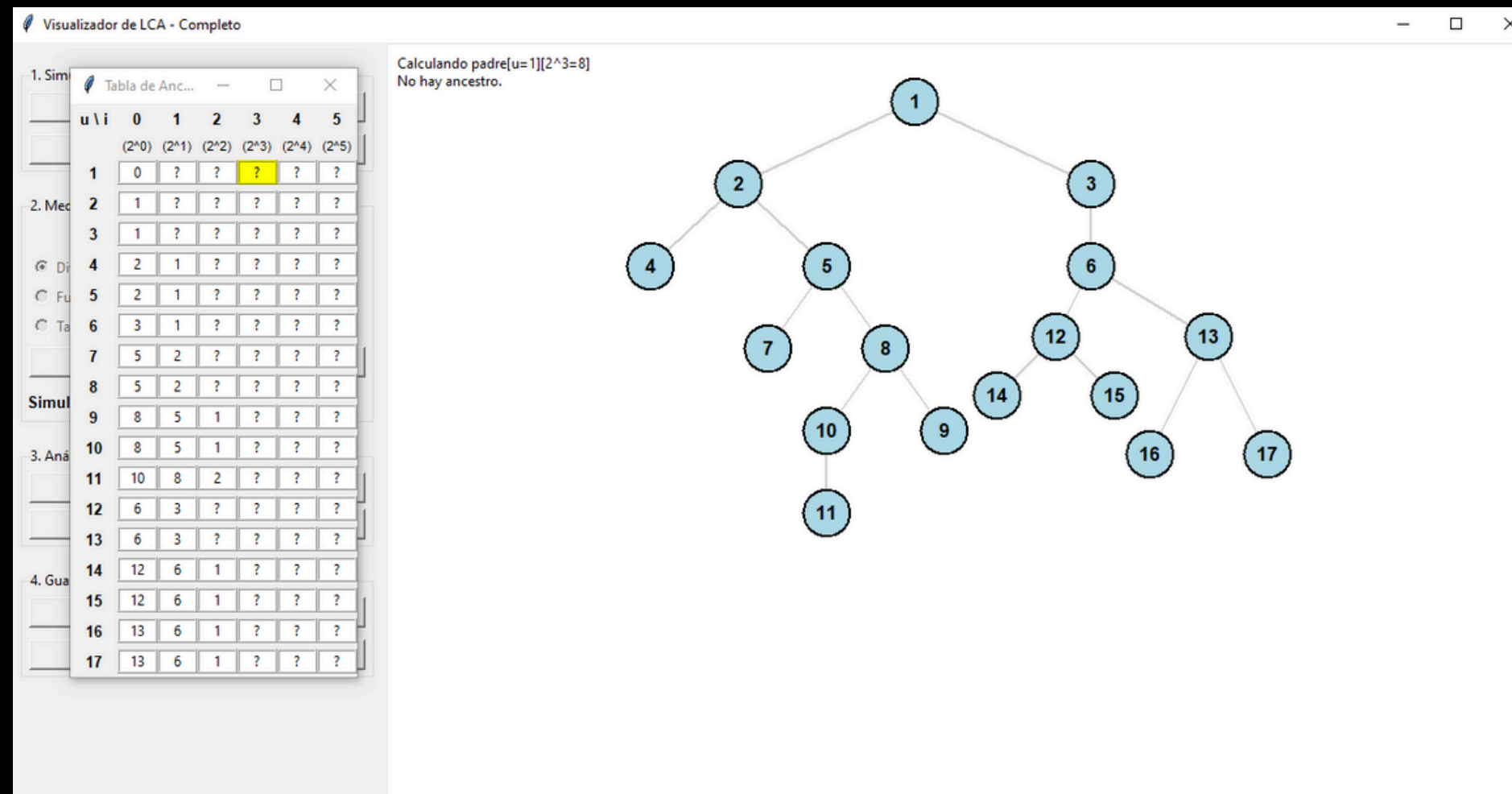
En nuestro programa, cuando se presiona “Calcular, Medir, Simular” con la opción de fuerza bruta, se muestra el recorrido que hace desde los nodos hasta el LCA.



LCA CON BINARY LIFTING

DIVIDE Y VENCERÁS CON PROGRAMACIÓN DINÁMICA

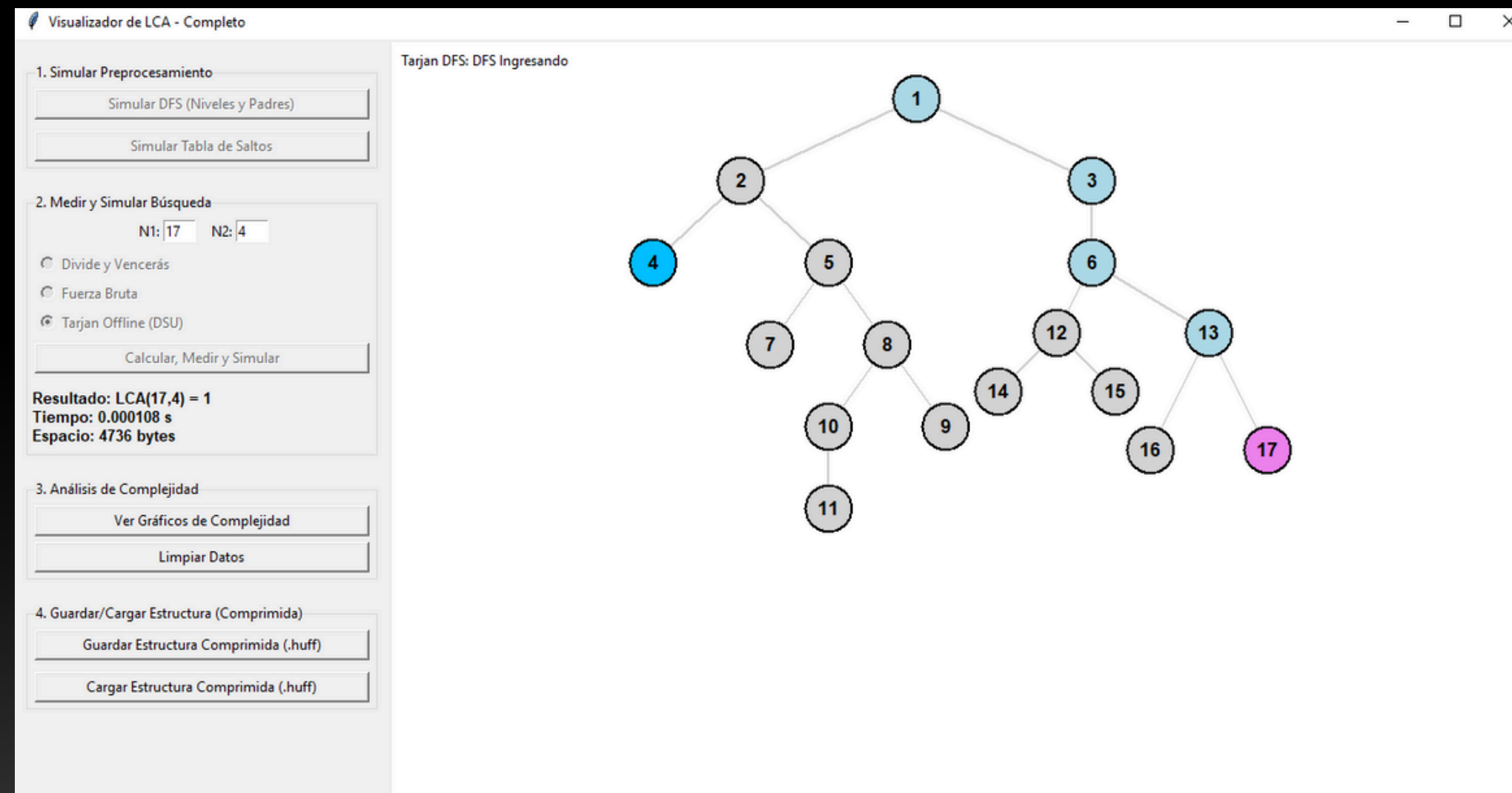
Se hace una etapa de preprocesamiento y se precalcula una tabla $\text{padre}[u][i]$ que almacena el ancestro de u a una distancia de 2^i . Esto permite dar "saltos" grandes hacia la raíz y se encuentra el LCA subiendo en potencias de 2 desde los nodos seleccionados.



TARJAN

Este algoritmo de grafos utiliza la estructura de datos Union-Find (Conjuntos Disjuntos). Tiene una complejidad casi lineal $O(N + Q\alpha(N))$, donde α es la inversa de la función de Ackermann.

Al presionar “Calcular, Medir, Simular” con la opción de Tarjan Offline, se puede ver el recorrido que se hace desde la raíz con un DFS, cuando llega a uno de los nodos a buscar lo marca, sigue y al llegar al final se regresa a la bifurcación inmediata y vuelve continuar con el DFS.



COMPARACIÓN DE DESEMPEÑO Y RESULTADOS

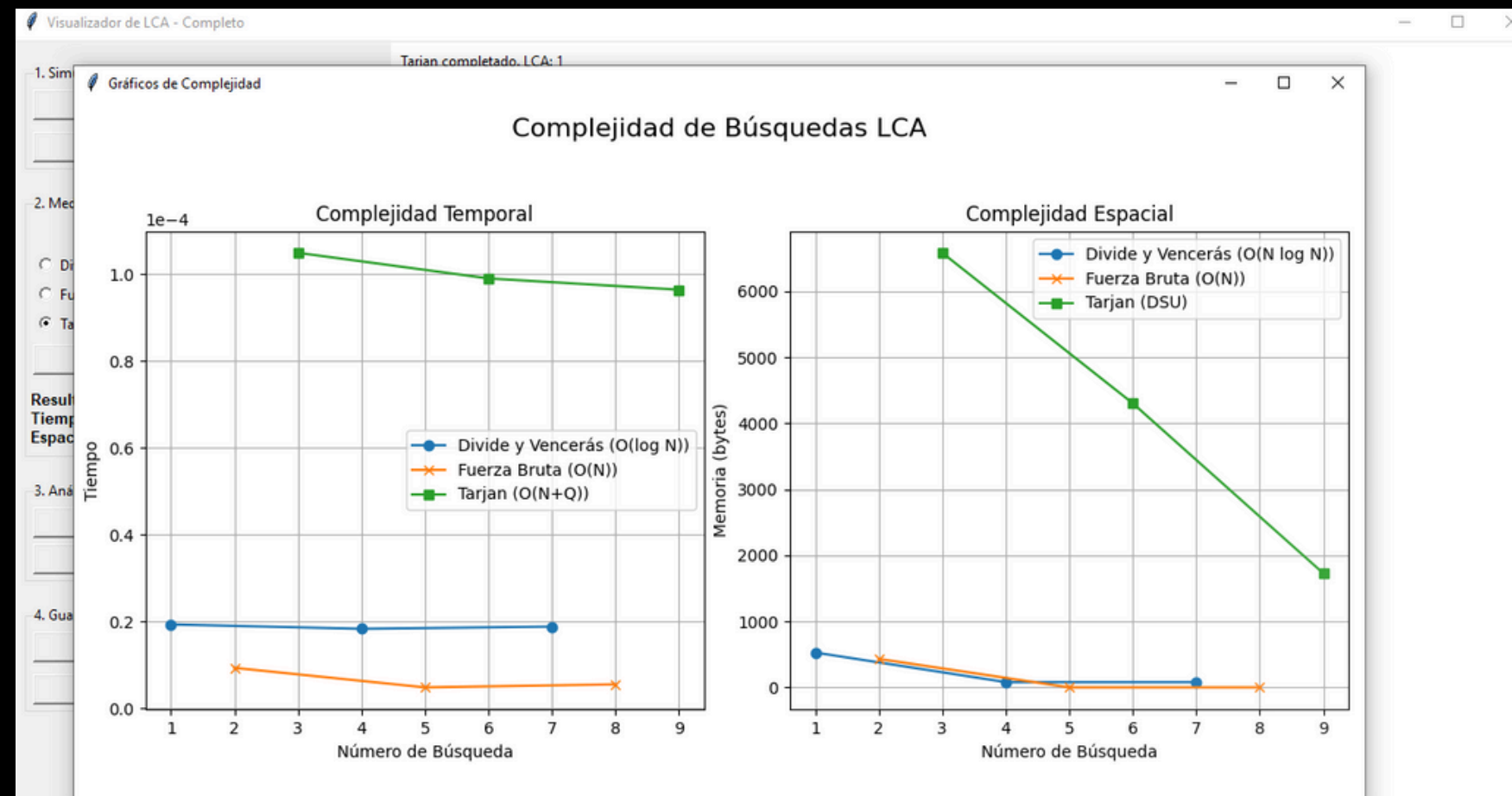
Utilizamos las librerías matplotlib y tracemalloc para generar gráficas en tiempo real.

1. Tiempo de Ejecución:

- **Binary Lifting:** Rápido para las consultas $O(\log N)$, pero requiere preprocesamiento.
- **Fuerza Bruta:** Su tiempo crece linealmente con la profundidad de los nodos. Lento en rutas largas.
- **Tarjan:** Muy eficiente, resolviendo la consulta durante un único recorrido DFS completo.

2. Consumo de Memoria:

- **Binary Lifting:** Consume más memoria debido a la tabla de saltos de tamaño $N * \log N$.
- **Fuerza Bruta:** Consume menos memoria (solo necesita punteros al padre, prácticamente nula).
- **Tarjan:** Requiere estructuras auxiliares para el DSU (padre, rango, ancestro).



CONCLUSIONES

El desarrollo de los programas nos ha permitido ver que no existe un algoritmo perfecto para cualquier caso, sino que depende de las restricciones y el contexto del problema:

1. **Si el árbol es estático y se realizarán muchas de consultas:** Binary Lifting por su velocidad de consulta logarítmica, a pesar de usar más memoria.
2. **Si el espacio de memoria es importante y el árbol no es profundo:** Fuerza Bruta por su implementación simple.
3. **Si se conocen todas las consultas con anterioridad:** Tarjan ofrece un rendimiento casi lineal.

Además, el añadir Huffman nos muestra que también se pueden comprimir estructuras como grafos.

En general, la interfaz gráfica nos facilita la comprensión de todos los métodos pero en especial conceptos poco complendibles como el binary lifting y la unión de los conjuntos en Tarjan.

REFERENCIAS

- Aprende Programación Competitiva. (2022, octubre 24). Olimpiada-informatica.org.
<https://aprende.olimpiada-informatica.org/algoritmia-lca>
- Codificación Huffman. (2024, junio 12). StudySmarter ES.
<https://www.studysmarter.es/resumenes/ciencias-de-la-computacion/representacion-de-datos-en-ciencias-de-la-computacion/codificacion-huffman/>
- Tarjan's off-line lowest common ancestors algorithm. (2016, mayo 18). GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/tarjans-off-line-lowest-common-ancestors-algorithm/>



**MUCHAS GRACIAS
POR SU ATENCIÓN**